EVALUATION RUNBOOK

Wiring In The Eval Loop

Not a specific submission's setup — this is the repeatable process: given any Task #2 repo you've already cloned, get rag-local-eval-loop attached to it and check the real values it produces.

ASSUMES ALREADY DONE

<team-name>'s repo already cloned locally, somewhere of your choosing.

On hand before you start:

- Python 3.10+ — the eval loop's own runtime, regardless of what the submission is written in.
- An OpenAI or Anthropic key — the eval loop's own judge, unrelated to whatever the submission uses.
- The submission running locally — its own README has that; the eval loop needs it reachable, not its source.

01 Clone the eval loop once, reuse it for every submission

One clone, anywhere convenient — you don't re-clone this per participant, only copy two small pieces out of it below.

```
git clone https://github.com/BeaconBandhu/rag-local-eval-loop.git
C:\tools\rag-local-eval-loop
```

02 Copy the eval loop into that submission's folder

This is the actual "attach it" step — drop eval/ and the launcher straight into the submission's own root, next to its own files. The tool finds itself there automatically; no path flag needed once you run it from inside this folder.

```
$sub = "C:\evals\task2-submissions\<team-name>"
Copy-Item C:\tools\rag-local-eval-loop\eval "$sub\eval" -Recurse
Copy-Item C:\tools\rag-local-eval-loop\run.ps1 "$sub\run.ps1"
```

ALTERNATIVE

Prefer not to touch the submission's own folder at all? Skip this step and pass --rag-root <team-name>'s path on every run instead — same result, nothing copied.

03 Give it a Python environment

A fresh one, right in the submission's folder — the launcher looks for .venv\Scripts\python.exe in whatever directory it's run from.

```
cd $sub
python -m venv .venv
.venv\Scripts\pip install -r C:\tools\rag-local-eval-loop\requirements.txt
```

IF NATIVE PYTHON

Also install the submission's own requirements.txt into this same .venv — native mode really imports and runs its code in-process, so it needs its own real dependencies too.

04 Tell it what kind of target this one is

Every submission needs exactly one of these two — check first whether the repo has Python embed()/generate_answer() functions to import, or whether it's only reachable as a running service.

A · Native Python module

Has real app/embedder.py + app/generator.py (or equivalent) to import directly.

```
# only if it's not the default app.embedder / app.generator:
$env:EVAL_EMBEDDER_MODULE  = "main"
$env:EVAL_GENERATOR_MODULE = "main"
```

B · HTTP service (Node, Go, anything non-Python)

Write a small JSON config mapping its real API onto what the eval loop expects — copy examples/http_target_configs/goarag.json as a starting template.

```
$env:EVAL_EMBEDDER_MODULE  = "eval.http_target"
$env:EVAL_GENERATOR_MODULE = "eval.http_target"
$env:EVAL_HTTP_CONFIG = "<path to that config>.json"
```

05 Set the judge credential

Faithfulness and correctness both need their own LLM judge, completely separate from anything the submission itself calls. Without this, both just report SKIPPED instead of failing the run.

```
$env:OPENAI_API_KEY = "sk-..."     # or ANTHROPIC_API_KEY
```

06 Smoke-test before spending real judge calls

Small and cheap — confirms every piece of wiring above actually works for this specific submission.

```
cd $sub
.\run.ps1 --num-answerable 3 --num-unanswerable 3 --workers 1
```

IF IT FAILS HERE

The error names the exact missing module, function, or unreachable endpoint — never a bare stack trace. Fix that one thing for this submission and rerun.

07 Run it for real, then check the values

Scale the sample size up once the smoke test is clean.

```
.\run.ps1 --num-answerable 50 --num-unanswerable 50
```

Prints the full report straight to the console, and saves the same data to results/<timestamp>.json — the second copy is what lets you diff one submission's numbers against another's later.

CHECKING THE VALUES

What each number in the report actually means

Five independent checks, every one scored against a real MSMARCO-XI row — nothing hand-written, and every check that can't run reports SKIPPED with a plain reason rather than a fake number.

RETRIEVAL	Recall@1/3/5 and MRR — does the submission's own embedding find the right passage among the sampled candidates.
FAITHFULNESS	Hallucination rate, judged against only the context it actually retrieved — never the ground-truth answer.
CORRECTNESS	Judged against MSMARCO-XI's real reference answer for that query.
RELIABILITY	The "lying factor" — on unanswerable queries, did it confidently fabricate instead of declining. This is the one that most directly catches a team overselling its guardrails.
LATENCY	P50/P70/P100 embed, search, and generation timing, measured from real requests — not whatever number the team's own README claims.

Reusable across every submission: steps 01–03 happen once per submission's folder, step 04 is the only branch point, and eval/http_target.py's own docstring has the full config schema for branch B.